

Reviewer Pack — Low-Degree Fourier Mass of Cut Indicator Bounds Tseitin Reso...

Entry ff19cf66644d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 07:33:32 UTC

Low-Degree Fourier Mass of Cut Indicator Bounds Tseitin Resolution

Entry ID: ff19cf66644d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 07:33:32 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic / hypercontractivity for boolean functions (Bonami-Beckner low-degree concentration of cut indicators on the boolean cube $\{0,1\}^V$, in the spirit of Friedgut-Kalai-Linial junta theorems) **Field B** (complexity object): Resolution refutation length $L_R(\text{Tseitin}(G,\sigma))$ of 3-regular Tseitin formulas with odd vertex charge σ (the Urquhart / Ben-Sasson-Wigderson canonical sub-Frege target)

Statement:

Let G be a connected 3-regular graph on n vertices and $m=3n/2$ edges, and let $\sigma \in \mathbb{F}_2^V$ have odd total parity. Define the cut-threshold indicator $f_G: \{0,1\}^V \rightarrow \{0,1\}$ by $f_G(x)=1$ iff $|E(S_x, V \setminus S_x)| \leq m/3$, where $S_x = \{v: x_v = 1\}$, and let its level- $\leq k$ Fourier mass be $W_{\leq k}[f_G] = \sum_{|S| \leq k} \hat{f}_G(S)^2$. Define $\nu(G) := -\log_2 W_{\leq \lfloor n/8 \rfloor}[f_G] - \log_2(1/E[f_G] + 1)$. The conjecture: $\log_2 L_R(\text{Tseitin}(G,\sigma)) \geq c \cdot \nu(G)$ for an absolute constant $c \geq 1/12$, while $\nu(G) = O(1)$ whenever G admits a vertex separator of size $O(1)$ with $O(1)$ edge boundary; a single 3-regular instance on $n \leq 40$ violating $\log_2 L_R \geq (1/12) \cdot \nu(G)$ refutes the bound.

Rationale (proposer's reasoning):

On expander G , cuts are sharply concentrated near $m/2$ by Bonami-Beckner hypercontractivity applied to the $(n$ -junta) cut function, so f_G is a near-zero, low-influence boolean function and its low-degree mass $W_{\leq \lfloor n/8 \rfloor}$ is exponentially small, forcing $\nu(G) = \Omega(n)$ and matching the Urquhart $2^{\Omega(n)}$ resolution lower bound. On non-expanders the small-cut event is realized by a constant-size junta of separator vertices, so f_G is essentially a low-degree function and $W_{\leq \lfloor n/8 \rfloor}$ is $\Omega(1)$, driving $\nu(G) = O(1)$. This bridges the Friedgut-Bourgain low-degree-mass / junta machinery (rarely applied to proof complexity) directly to Tseitin resolution via the cut indicator, without invoking spectral λ_2 or vertex expansion explicitly.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fecbae42c36f30cf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 450 pooled 3-regular instances ($n \in \{10, 12, 14, 16, 18\}$, 30 seeds $\times$ 3 families), conjecture is SUPPORTED iff Spearman $\rho(v(G), \log_2 \# \text{decisions}) \geq 0.6$ AND every instance with $v(G) \geq n/4$ satisfies $\log_2(\# \text{decisions}) \geq v(G)/12$. A single instance with $v(G) \geq n/4$ and $\log_2(\# \text{decisions}) < v(G)/12$ FALSIFIES.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Tseitin formulas resolution lower bounds expansion Fourier - low-degree Fourier concentration cut indicator boolean cube hypercontractivity - Ben-Sasson Wigderson resolution width Tseitin expander Fourier analysis

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ramanujan_graph(n):
        # Configuration model to generate a 3-regular graph
        G = [[] for _ in range(n)]
        degrees = [0] * n
        while any(d != 3 for d in degrees):
            u, v = random.sample(range(n), 2)
            if u not in G[v] and v not in G[u]:
                G[u].append(v)
                G[v].append(u)
                degrees[u] += 1
                degrees[v] += 1
        return G

    def generate_planted_separator_graph(n):
        # Two K3,3 blocks joined by a small bridge
        G = [[] for _ in range(6)]
        G[0], G[1], G[2] = [1, 3, 4], [0, 2, 5], [1, 2]
```

```

G[3], G[4], G[5] = [0, 5], [0, 4], [1, 4]
return G

def generate_cycle_gadgets_graph(n):
    # Cycle of K4 gadgets
    G = [[] for _ in range(4 * n)]
    for i in range(n):
        G[4 * i] = [4 * (i + 1) % (4 * n), 4 * (i - 1) % (4 * n), 4 * i + 1, 4 * i + 2]
        G[4 * i + 1] = [4 * i, 4 * (i + 1) % (4 * n), 4 * i + 3]
        G[4 * i + 2] = [4 * i, 4 * (i - 1) % (4 * n), 4 * i + 3]
        G[4 * i + 3] = [4 * i, 4 * (i + 1) % (4 * n)]
    return G

def walsh_hadamard_transform(f):
    n = len(f)
    for s in range(1, n):
        mask = 1 << s
        for i in range(n // (2 ** s)):
            for j in range(2 ** s):
                f[i * (2 ** s) + j] += f[i * (2 ** s) + j + mask]
    return f

def compute_f_hat(G, sigma):
    n = len(G)
    f_hat = [0] * (1 << n)
    for i in range(1 << n):
        S = [j for j in range(n) if i & (1 << j)]
        count = sum(len([v for v in G[j] if v not in S]) <= len(G) // 3 for j in S)
        f_hat[i] = (-1) ** count
    return walsh_hadamard_transform(f_hat)

def compute_nu(G):
    n = len(G)
    f_hat = compute_f_hat(G, [0] * n)
    nu = -math.log2(sum(f_hat[i] ** 2 for i in range(1 << (n // 8 + 1)))) - math.log2(1 / sum(
    return nu

def compute_log2_LR(G, sigma):
    # Simple DPLL refutation count for Tseitin formula
    n = len(sigma)
    stack = [(0, [])]
    decisions = 0
    while stack:
        i, assignment = stack.pop()
        if i == n:
            return math.log2(decisions)
        if assignment[i] is None:
            assignment[i] = 0
            stack.append((i + 1, assignment[:]))
            assignment[i] = 1
            stack.append((i + 1, assignment[:]))
        else:
            decisions += 1
    return math.log2(decisions)

def generate_instance(n):

```

```

    if n == 10: return generate_ramanujan_graph(10)
    elif n == 12: return generate_planted_separator_graph(12)
    elif n == 14: return generate_cycle_gadgets_graph(14)
    else: raise ValueError("Unsupported graph type")

def run_experiment(n):
    G = generate_instance(n)
    sigma = [random.randint(0, 1) for _ in range(len(G))]
    nu = compute_nu(G)
    log2_L_R = compute_log2_L_R(G, sigma)
    return {
        "metric_name": "nu_G",
        "metric_value": nu,
        "instances_tested": 1,
        "conjecture_holds": nu >= n / 4 and log2_L_R >= nu / 12,
        "counterexample": "" if nu < n / 4 or log2_L_R >= nu / 12 else f"nu_G={nu}, log2(L_R)="
    }

results = [run_experiment(n) for n in {10, 12, 14}]
mean_nu = sum(r["metric_value"] for r in results) / len(results)
std_nu = math.sqrt(sum((r["metric_value"] - mean_nu) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_nu": mean_nu,
    "std_nu": std_nu,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_nu = sum(r["mean_nu"] for r in results) / len(results)
    std_nu = math.sqrt(sum((r["mean_nu"] - mean_nu) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["support_fraction"] >= 0.8) / len(results)

    if all(r["support_fraction"] >= 0.8 for r in results):
        print(f"RESULT: SUPPORTED mean={mean_nu} std={std_nu} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if r["support_fraction"] < 0.8), None)
        print(f"RESULT: FALSIFIED counterexample='support_fraction<0.8' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s without producing any RESULT line, so the pre-registered support condition (Spearman $\rho \geq 0.6$ plus per-instance bound) could not be evaluated. Per the rules, a crash/timeout yields INCONCLUSIVE. | next: Reduce the instance pool (e.g., $n \in \{10,12,14\}$, 10 seeds $\times$ 3 families) and cap the SAT solver’s decision budget per instance so the 450-instance sweep finishes within the timeout.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	154635
2	preregistration	claude_max	opus	0	0	6962
3	novelty	claude_max	opus	0	0	4019
4	novelty	claude_max	opus	0	0	9091
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	115461
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20151
7	judge	claude_max	opus	0	0	7391

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 317711 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/ff19cf66644d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ff19cf66644d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ff19cf66644d.tar.gz` (if generated)